

Reflected XSS

반사형 XSS를 통한 클라이언트 토큰 검증 우회 (Reflected XSS to Client-Side Token Comparison Bypass)

TARGET problem09	PREPARED FOR 8low8lowme Team	SEVERITY: MEDIUM
AUTHOR 김혜미 (설계) / 구현팀	CWE CWE-79 · CWE-602	STATUS: FINAL

1. 개요 (Executive Summary)

2. 공격 시나리오 (Attack Narrative)

단계	기법	확보한 정보 / 결과
1	웹 페이지 접속 후 입력창에 <code><script>alert(1)</script></code> 테스트	입력값이 그대로 반사되어 스크립트가 실행됨 (Reflected XSS 확인)
2	개발자 도구에서 <code>document.cookie</code> 확인	<code>ctf_token</code> 쿠키에 인코딩된 문자열이 담겨 있음을 발견
3	힌트(덧셈암호→니블스왑→뒤집기→커스텀 base64)를 역순으로 적용하는 디코더 작성	쿠키값을 디코딩해 원본 토큰(raw token) 획득
4	입력창에 <code><script>findFlag('원본토큰')</script></code> 삽입	reflected XSS로 스크립트 실행, <code>findFlag()</code> 가 쿠키를 내부적으로 재검증
5	비교 성공 시 화면에 노출된 FLAG 확인	FLAG 텍스트가 <code>#flag</code> 영역에 출력됨

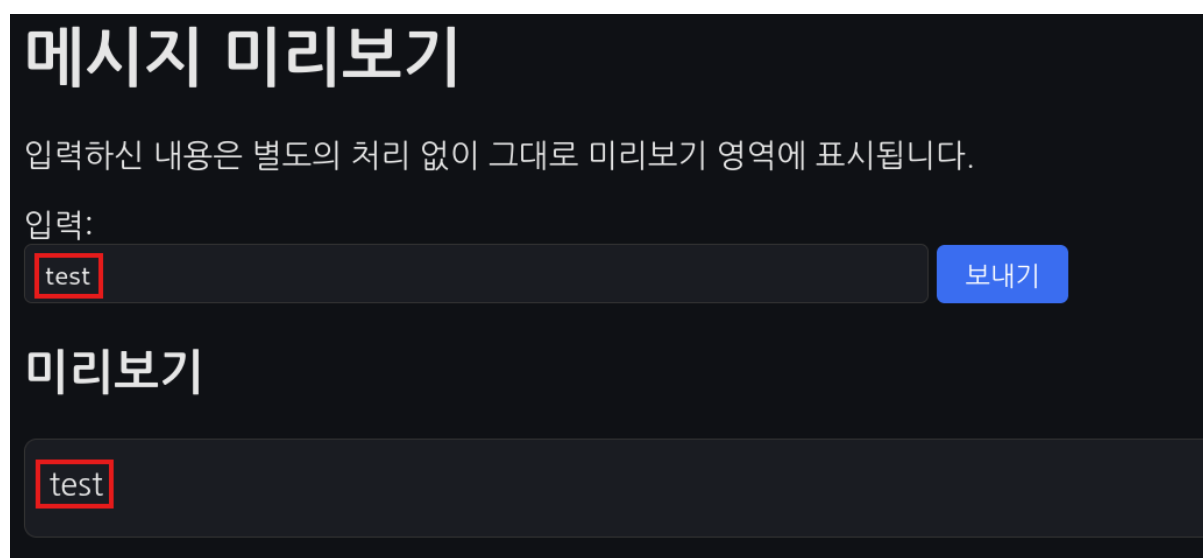
3. 상세 재현 절차 (Proof of Concept)

3.1 정찰 — 페이지 구조 및 반사 지점 확인

problem09를 분석해보니, 사용자가 입력한 값을 서버가 아무 처리 없이 그대로 화면에 출력하고 있었다. 그래서 입력창에 스크립트를 넣으면 그대로 실행되는 반사형 XSS가 있었다.

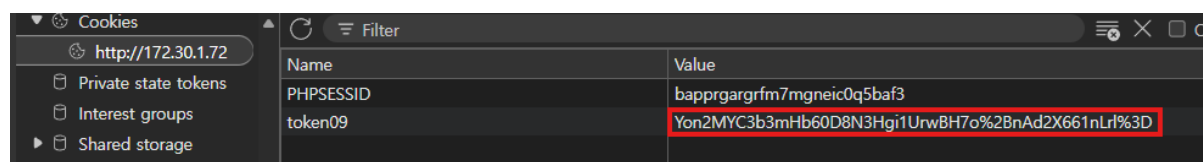
원래 토큰 값 자체는 인코딩되어 쿠키에만 담겨 있어 눈으로 보서는 알 수 없다. 문제는 이 토큰이 맞는지 확인하는 로직(findFlag)이 서버가 아니라 브라우저(클라이언트 자바스크립트) 안에 그대로 들어있다는 점이다. 그래서 방금 찾은 XSS로 스크립트를 실행시켜서 이 검증 함수를 직접 호출하면, 정상적인 절차를 거치지 않고도 원하는 값으로 검증을 통과시킬 수 있었다.

정리하면, "입력값을 그대로 출력하는 XSS 취약점"과 "중요한 판단을 브라우저에만 맡긴 설계"가 합쳐지면서, 인증을 우회할 수 있는 문제로 이어진 것이다.



3.2 쿠키 분석 — 인코딩된 토큰 확인

개발자 도구(F12) Application/저장소 탭 또는 콘솔에서 document.cookie를 확인했다. token 값이 평문이 아닌 알아볼 수 없는 문자열로 인코딩되어 있음을 확인했다.



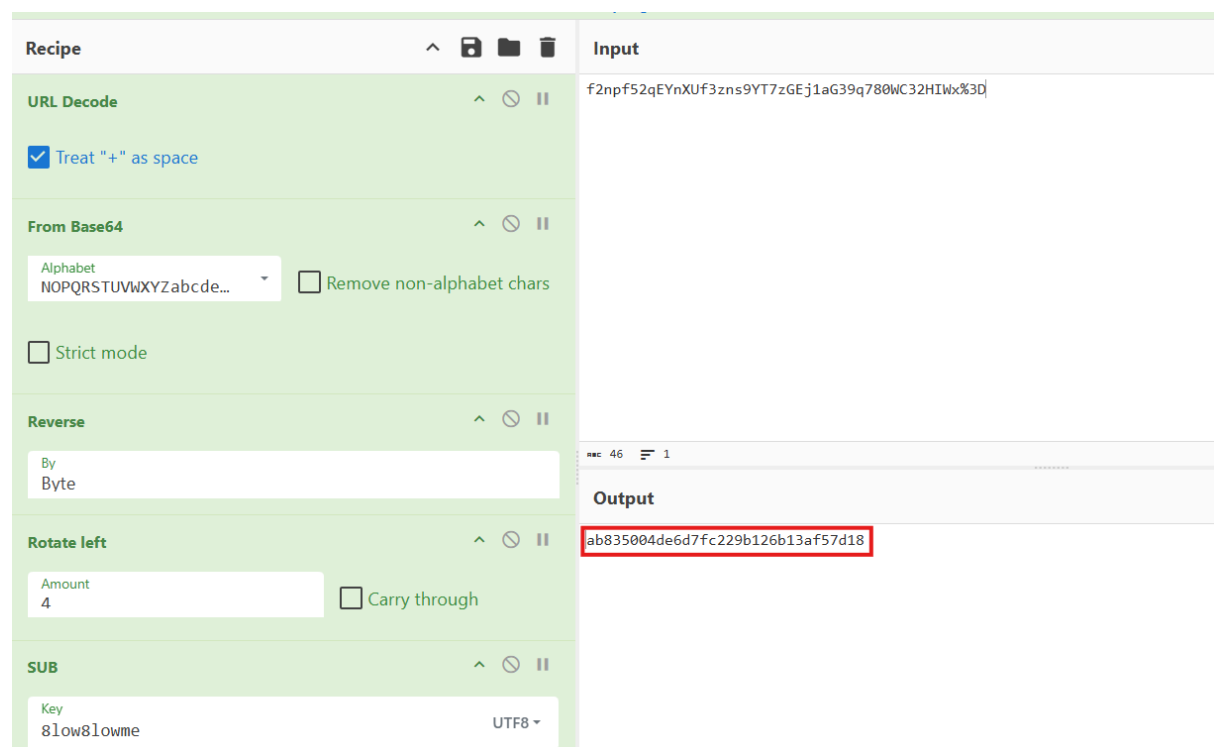
3.3 디코딩 스크립트 작성 — 원본 토큰 복원

문제 설명에는 별도 힌트가 없어, 개발자 도구 Sources 탭에서 script.js를 직접 열람했다. 코드를 분석해 인코딩 순서(덧셈 암호 → 니블 스왑 → 뒤집기 → 커스텀 base64)와 KEY값 ("8low8lowme"), 13칸 회전된 커스텀 base64 알파벳을 확인했다. 이를 역순으로 적용하는 디코더를 Python으로 직접 작성했다. (CyberChef 등 온라인 도구로도 동일하게 재현 가능하다.)

```
# 커스텀 base64 해제 -> 뒤집기 -> 니블 스왑 -> 덧셈 역연산(mod 256)
KEY = b"8low8lowme"
STD = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
CUSTOM = STD[13:] + STD[:13]

def decode(cookie_val):
    raw = base64.b64decode(cookie_val.translate(str.maketrans(CUSTOM, STD)))
    raw = raw[::-1]
    raw = bytes(((b << 4) | (b >> 4)) & 0xFF for b in raw)
    return bytes((b - KEY[i % len(KEY)]) & 0xFF for i, b in enumerate(raw)).decode()
```

이 스크립트로 쿠키값을 디코딩해 원본 세션 토큰(raw token)을 얻었다.



3.4 XSS 페이로드 삽입 — 클라이언트 검증 우회

디코딩해서 얻은 원본 토큰을, findFlag() 함수를 호출하는 스크립트에 담아 입력창에 넣었다. findFlag()는 브라우저(JS) 안에서 쿠키값을 다시 디코딩해서, 내가 넘긴 값이랑 같은지 비교하는 함수다. 이 비교가 서버가 아니라 브라우저에서 일어나기 때문에, XSS로 이 함수를 직접 호출하면 정상적인 절차 없이도 비교를 통과시킬 수 있었다.

```
<script>findFlag('여기에_디코딩한_원본_토큰')</script>
```

3.5 FLAG 획득

삽입한 스크립트가 반사되어 실행되면서 findFlag()가 호출되었고, 토큰 비교에 성공해 화면에 FLAG가 노출되었다.

메시지 미리보기

입력하신 내용은 별도의 처리 없이 그대로 미리보기 영역에 표시됩니다.

입력:

```
<script>findFlag('ab835004de6d7fc229b126b13af57d18')</script>
```

보내기

미리보기

FLAG: FLAG{I2lyOGU4JmdfNWR1OA}

4. 근본 원인 분석 (Root Cause)

- 사용자 입력(q 파라미터)이 이스케이프 없이 그대로 HTML에 반영되어 Reflected XSS가 발생함
- 토큰 검증(findFlag) 로직이 서버가 아닌 클라이언트 JavaScript 안에서 수행되어, 브라우저에서 열람·우회가 가능함
- 원본 토큰 자체는 직접 노출하지 않지만, 인코딩/디코딩 알고리즘이 클라이언트 코드에 그대로 포함되어 있어 역산이 가능함
- 서버는 사용자가 화면에서 무엇을 봤는지에 대한 별도 검증 없이, 클라이언트 측 판단 결과를 그대로 신뢰하는 구조임